UNIVERSITY OF GHANA

DEPARTMENT OF COMPUTER SCIENCE

GHANA SMART HEALTH SERVICE OPERATIONS OPTIMIZER

(GH-Health-System)

Prepared by: **InnerGroup**

Comprehensive Technical Architecture, Custom Data Structures, Algorithmic Analysis & Systems Documentation Report

Course Module & Team Mandate: DCIT 204 / DCIT 308 – Data Structures & Algorithms I & II. Developed by Team InnerGroup as an enterprise-grade semester project demonstrating data structure design, graph pathfinding, algorithmic efficiency, SQLite persistence, and runtime benchmarking.

Metadata Attribute	Details
Project Name	Ghana Smart Health Service Operations Optimizer (GH-Health-System)
Development Team	InnerGroup
Institution	University of Ghana, Legon
Course Code	DCIT 204 / DCIT 308: Data Structures & Algorithms I & II
Target System	Accra & Regional Ghanaian Health Network (Korle-Bu, Ridge, 37 Military)
Core Runtime	Java 17+ / OpenJDK 25, SQLite JDBC 3.46.0.0, SLF4J 2.0.16
Documentation Date	Academic Year 2025/2026
Document Version	Version 1.1 (Team InnerGroup Final Documentation)

Table of Contents

Section #	Section Title	Key Content Covered
1.0	Executive Summary & System Objectives	Healthcare operational challenges, Team InnerGroup scope, first-principles mandate.
2.0	System Architecture & Directory Hierarchy	Layered software architecture, package breakdown, file structure.
3.0	Domain Data Models & Schema Design	Entity models (Location, Road, Resource, ServiceRequest) & enumerations.
4.0	Custom Data Structures & Complexity Analysis	15 custom data structures, implementation mechanics & Big-O time/space matrix.
5.0	Algorithmic Implementations & Operational Logic	Dijkstra, Kruskal MST, Quick/Merge/Heap sort, Greedy & Knapsack optimization.
6.0	Database Engine & Persistence (SQLite)	SQLite connection pooling, DAO layers, CSV dataset seeders.
7.0	Service Layer & Business Operations	RoutingService, SchedulingService, OptimizationService, ReportService.
8.0	Performance Profiling & Testing Matrix	Runtime measurement, heap memory tracking, JUnit test suite inventory.
9.0	Developer Setup, Compilation & Build Guide	Maven configuration, library manual installation, IntelliJ setup, Git workflow.

1. Executive Summary & System Objectives

The Ghana Smart Health Service Operations Optimizer (GH-Health-System) was engineered by Team InnerGroup to resolve operational bottlenecks in Ghanaian emergency response and hospital resource distribution. Major referral hubs such as Korle-Bu Teaching Hospital, Ridge Hospital, 37 Military Hospital, and outlying regional clinics frequently face challenges regarding ambulance response times, emergency bed saturation, and specialist allocation.

Core Architectural Mandate: In strict compliance with DCIT 204/308 course requirements, Team InnerGroup designed and implemented all foundational data structures (dynamic arrays, linked lists, stacks, queues, priority queues, hash tables, binary search trees, red-black trees, disjoint sets, and graph adjacency lists) from first principles in pure Java without using built-in Java Collection Framework classes (such as java.util.ArrayList, LinkedList, HashMap, or PriorityQueue) for core operations.

1.1 Primary System Objectives

- • Emergency Transit Optimization: Calculating real-time shortest paths for ambulances navigating urban traffic and varied road conditions across Ghanaian municipalities.
- • Priority-Driven Dispatch: Scheduling emergency service requests using priority queues weighted by patient condition urgency and elapsed waiting time.
- • Resource Load Balancing: Dynamically matching available ICU beds, ventilators, and surgical specialists to high-density emergency wards.
- • Embedded Relational Storage: Ingesting CSV datasets (locations, roads, resources) into a local SQLite database for transactional persistence.
- • Empirical Benchmarking: Executing automated runtime profiling (nanosecond resolution) and JVM heap memory benchmarking to evaluate theoretical Big-O expectations.

2. System Architecture & Directory Hierarchy

Team InnerGroup structured GH-Health-System around a modular, multi-tiered architecture that separates user interface interactions, business domain services, algorithmic execution, custom data structures, and database persistence layers.

2.1 Layered Architecture Pattern

1. UI Layer (com.ug.optimizer.ui): Interactive command-line interface (ConsoleMenu, MenuHandler) handling dispatcher inputs and rendering operational reports.

2. Service Layer (com.ug.optimizer.services): High-level business logic orchestrators (RoutingService, SchedulingService, OptimizationService, ReportService) coordinating resource allocation and route calculation.

3. Algorithm Layer (com.ug.optimizer.algorithms): Pure algorithmic implementations categorised into graph pathfinding, sorting, searching, greedy, and dynamic programming.

4. Data Structure Layer (com.ug.optimizer.datastructures): First-principles data structures providing underlying storage and retrieval mechanics.

5. Database & Data Access Layer (com.ug.optimizer.database): SQLite connection managers, Data Access Objects (DAOs), CSV dataset seeders, and seed generators.

6. Domain Model Layer (com.ug.optimizer.model): Encapsulated domain entities and enum definitions.

2.2 Repository File Layout

Package / Directory	Component Description	Primary Files & Role
com.ug.optimizer.algorithms	Algorithms Suite	Searching.java, Graph Algorithms, Sorting, Greedy, DP
com.ug.optimizer.database	Database & DAOs	DatabaseConnection.java, LocationDAO, RoadDAO, ResourceDAO, DataSeeder
com.ug.optimizer.datastructures	Custom Data Structures	MyArrayList, MyLinkedList, MyPriorityQueue, MyHashMap, MyBST, Graph, etc.
com.ug.optimizer.model	Domain Entities & Enums	Location.java, Road.java, Resource.java, ServiceRequest.java, Enums
com.ug.optimizer.performance	Profiling & Benchmarking	Benchmark.java, MemoryProfiler.java, RuntimeRecorder.java
com.ug.optimizer.services	Business Logic Services	RoutingService.java, SchedulingService.java, OptimizationService.java
com.ug.optimizer.ui	Console UI Package	ConsoleMenu.java, MenuHandler.java
data/	CSV Seed Datasets	locations.csv (53 sites), resources.csv (31 items), roads.csv (98 routes)

test/java/	Automated Unit Tests	20+ JUnit test suites covering data structures, DAOs, and algorithms

3. Domain Data Models & Schema Design

The domain model layer captures the real-world characteristics of health network infrastructure, patient requests, and operational metrics.

3.1 Domain Entity Specifications

Model Class	Attributes & Data Types	Business Domain Meaning
Location	id (int), name (String), area (String), locationType (LocationType), latitude (double), longitude (double)	Represents physical healthcare facilities, emergency wards, blood banks, or ambulance stations.
Road	fromLocationId (int), toLocationId (int), distance (double), travelTime (double), roadConditionWeight (double)	Models a directed road link between two locations, weighted by distance, travel delay, and surface condition.
Resource	id (int), type (String), homeLocationId (int), capacity (int), availabilityStatus (ResourceStatus)	Tracks available medical assets including ambulances, doctors, nurses, ICU beds, and ventilators.
ServiceRequest	requestId (int), patientId (int), locationId (int), urgencyLevel (UrgencyLevel), status (RequestStatus), timestamp (long)	Captures emergency medical calls, patient location, urgency level, and dispatch status.
AlgorithmRun	runId (int), algorithmName (String), inputSize (int), executionTimeNs (long), memoryUsedBytes (long)	Audit record measuring empirical algorithmic execution time and memory usage.
AuditEvent	eventId (int), eventType (String), description (String), timestamp (long)	Logs security events, system configuration changes, and operational user actions.

3.2 System Enumerations

• LocationType: EMERGENCY, PHARMACY, WARD, ICU, OPD, LABORATORY, RADIOLOGY, ADMINISTRATION, TRANSPORT, STAFF, RECORDS, BLOOD_BANK, DIALYSIS, ENDOSCOPY, CLINIC.

• UrgencyLevel: CRITICAL (Weight 4 - Immediate life threat), HIGH (Weight 3 - Severe condition), MEDIUM (Weight 2 - Moderate priority), LOW (Weight 1 - Non-urgent).

• ResourceStatus: AVAILABLE (Ready for dispatch), IN_USE (Currently deployed), UNDER_MAINTENANCE (Servicing), DISPATCHED (En route).

• RequestStatus: PENDING (In queue), DISPATCHED (Ambulance assigned), IN_PROGRESS (Treatment ongoing), COMPLETED (Resolved), CANCELLED (Revoked).

4. Custom Data Structures & Complexity Analysis

To meet the academic objectives of DCIT 204/308, 15 custom data structures were implemented from scratch by Team InnerGroup. Each structure handles memory layout, pointer references, dynamic array expansion, collision resolution, or tree rebalancing manually.

4.1 Summary of Custom Data Structures

1. Dynamic Lists: MyArrayList (resizable array with amortized O(1) appending) and MyLinkedList (doubly-linked list with head/tail pointers).

2. Stacks & Queues: MyStack (LIFO array stack), MyQueue (FIFO queue), MyCircularQueue (bounded ring-buffer queue), and MyDeque (double-ended queue).

3. Priority Queues: MyPriorityQueue (min/max binary heap supporting O(log n) insertion and extraction).

4. Associative Maps & Sets: MyHashMap (chaining hash table with load-factor resizing), MyMap (key-value pair map interface), and MySet (unique value collection).

5. Tree Structures: MyBST (Binary Search Tree), MyBTree (B-Tree for disk block indexing), and MyRedBlackTree (self-balancing binary search tree with color rotation).

6. Graph & Disjoint Set: Graph (adjacency-list representation of weighted directed graphs) and DisjointSet (Union-Find with path compression and rank optimization).

4.2 Big-O Algorithmic Complexity Matrix

Data Structure	Access (Avg/Worst)	Search (Avg/Worst)	Insert (Avg/Worst)	Delete (Avg/Worst)	Space	Operational Healthcare Use Case
MyArrayList	O(1) / O(1)	O(n) / O(n)	O(1)* / O(n)	O(n) / O(n)	O(n)	Storing static location records.
MyLinkedList	O(n) / O(n)	O(n) / O(n)	O(1) / O(1)	O(1) / O(1)	O(n)	Sequential emergency request logs.
MyStack	O(n) / O(n)	O(n) / O(n)	O(1) / O(1)	O(1) / O(1)	O(n)	Undo/redo operational buffer.
MyQueue	O(n) / O(n)	O(n) / O(n)	O(1) / O(1)	O(1) / O(1)	O(n)	First-come-first-served patient queue.
MyCircularQueue	O(1) / O(1)	O(n) / O(n)	O(1) / O(1)	O(1) / O(1)	O(k)	Fixed-capacity

						ambulance ring buffer.
MyPriorityQueue	O(1) min	O(n) / O(n)	O(log n)	O(log n)	O(n)	Urgency-weighted patient dispatch.
MyHashMap	N/A	O(1) / O(n)	O(1) / O(n)	O(1) / O(n)	O(n)	O(1) resource & doctor lookup.
MyBST	O(log n) / O(n)	O(log n) / O(n)	O(log n) / O(n)	O(log n) / O(n)	O(n)	Sorted patient ID searching.
MyRedBlackTree	O(log n)	O(log n)	O(log n)	O(log n)	O(n)	Balanced index of active emergency requests.
Graph	O(1)	O(V + E)	O(1)	O(V + E)	O(V+E)	Accra road network map modeling.
DisjointSet	O(1)	O(α(n))	O(1)	O(α(n))	O(n)	Kruskal MST network connectivity.

5. Algorithmic Implementations & Operational Logic

The algorithms suite implemented by Team InnerGroup contains optimized procedural implementations categorized into five distinct algorithmic paradigms:

5.1 Shortest Path & Graph Routing (Dijkstra's Algorithm)

Emergency ambulance dispatch requires computing the fastest path between an ambulance bay (e.g., Korle-Bu Ambulance Bay) and an incident scene. Dijkstra's algorithm is implemented using an adjacency-list Graph and custom MyPriorityQueue. Edge weights combine physical distance, average speed, and road condition parameters:

> **Edge Weight Formula:** Effective Weight = Distance (km) * (1.0 / Travel Speed) * Road Condition Weight. Severe road degradation increases effective traversal cost, routing ambulances around congested or damaged roads.

5.2 Minimum Spanning Tree (Kruskal's Algorithm)

To establish resilient tele-health connectivity and emergency supply links across health centers with minimum infrastructure cost, Kruskal's algorithm processes graph edges in ascending order of weight. Cycle detection is achieved in near O(1) amortized time using the custom DisjointSet class with path compression and rank optimization.

5.3 Priority Sorting Suite

• QuickSort: Dual-pivot partitioning sorting requests by composite urgency score.

• MergeSort: Stable O(n log n) divide-and-conquer algorithm used when patient request order stability must be preserved.

• HeapSort: In-place sorting utilizing binary min/max heap transformations.

5.4 Resource Allocation (Greedy & Dynamic Programming)

• Greedy Scheduling: Assigns nearest available ambulances to CRITICAL requests immediately.

• Knapsack DP Optimization: Solves 0/1 Knapsack for dispatching mobile pharmacy vans with restricted capacity to maximize medical relief value.

6. Database Engine & Persistence (SQLite)

GH-Health-System uses SQLite for zero-configuration, transactional embedded storage. The database schema stores physical location coordinates, road network connections, available resources, and active service requests.

6.1 Relational Database DDL Schema

CREATE TABLE locations (id INTEGER PRIMARY KEY, name TEXT, area TEXT, type TEXT, latitude REAL, longitude REAL);

CREATE TABLE roads (from_id INTEGER, to_id INTEGER, distance REAL, travel_time REAL, weight REAL, PRIMARY KEY (from_id, to_id));

CREATE TABLE resources (id INTEGER PRIMARY KEY AUTOINCREMENT, type TEXT, home_location_id INTEGER, capacity INTEGER, status TEXT);

CREATE TABLE service_requests (id INTEGER PRIMARY KEY AUTOINCREMENT, patient_id INTEGER, location_id INTEGER, urgency TEXT, status TEXT, timestamp INTEGER);

6.2 Data Access Objects (DAOs) & Data Seeder

DAO / Database Component	Methods & Operational Functionality
DatabaseConnection	Manages SQLite JDBC Connection instance (jdbc:sqlite:gh_health.db) with auto-commit control.
LocationDAO	insertLocation(), getLocationById(), getAllLocations(), deleteLocation().
RoadDAO	insertRoad(), getRoadsFromLocation(), getAllRoads(). Populates Graph adjacency list.
ResourceDAO	insertResource(), updateStatus(), getAvailableResourcesByType().
ServiceRequestDAO	createRequest(), updateRequestStatus(), getPendingRequestsByUrgency().
DataSeeder	Parses locations.csv, roads.csv, and resources.csv using CSVReader.java and populates database tables upon initial boot.

7. Service Layer & Business Operations

The service layer provides high-level APIs that orchestrate data structures, database DAOs, and algorithms to execute complete operational workflows.

7.1 RoutingService

Responsibility: Executes ambulance pathfinding between any two facilities in the Ghanaian health network. It retrieves the Graph model from RoadDAO, executes Dijkstra's algorithm, and constructs a step-by-step navigation itinerary containing total distance, estimated travel time, and intermediate checkpoints.

7.2 SchedulingService

Responsibility: Manages emergency request intake. Enqueues incoming requests into MyPriorityQueue based on UrgencyLevel. When an ambulance or medical specialist becomes AVAILABLE, SchedulingService dequeues the highest priority request and updates ResourceDAO and ServiceRequestDAO status to DISPATCHED.

7.3 OptimizationService

Responsibility: Runs global system optimizations. Periodically evaluates hospital bed saturation and redistributes non-emergency cases to nearby facilities with lower capacity utilization using Knapsack dynamic programming.

7.4 ReportService

Responsibility: Generates operational summary reports, calculates average ambulance response times, outputs resource allocation summaries, and formats audit trace logs for hospital administrators.

8. Performance Profiling & Testing Matrix

8.1 Automated Benchmarking Suite

The performance package (Benchmark.java, MemoryProfiler.java, RuntimeRecorder.java) measures theoretical vs empirical algorithm execution:

• Execution Time Measurement: Uses System.nanoTime() to record precise execution duration across varying dataset sizes (n = 100 to n = 10,000).

• Heap Memory Profiling: Invokes Runtime.getRuntime().totalMemory() - Runtime.getRuntime().freeMemory() before and after execution to measure byte allocation.

• Audit Reporting: Records performance metrics into AlgorithmRun objects and exports performance benchmark CSVs to results/graphs/.

8.2 Unit Testing Matrix

Team InnerGroup included full unit test suites inside test/java/ verifying correctness under boundary conditions:

Test Suite	Target Component	Key Test Scenarios Covered
MyArrayListTest	MyArrayList	Appending, dynamic expansion (2x resize), index bound checks, element removal.
MyLinkedListTest	MyLinkedList	Head/tail insertion, node deletion, iterator traversal, empty list operations.
MyPriorityQueueTest	MyPriorityQueue	Min-heap property validation, priority updates, empty queue extraction.
MyHashMapTest	MyHashMap	Hash collision handling, rehash at load factor 0.75, key lookup, null handling.
MyBSTTest & MyBTreeTest	MyBST / MyBTree	In-order traversal, binary search insertion, node deletion, root balancing.
GraphTest & DisjointSetTest	Graph / DisjointSet	Dijkstra path correctness, cycle detection in Kruskal, union by rank.
SearchAlgorithmsTest	Searching	Binary search on sorted arrays, linear search fallback, missing key return -1.

9. Developer Setup, Compilation & Build Guide

9.1 Prerequisites

• Java Development Kit (JDK 17 or JDK 25 Hotspot).

• Apache Maven 3.8+ (for dependency resolution).

• IntelliJ IDEA / Eclipse IDE (optional).

9.2 Dependency Configuration (pom.xml)

Ensure pom.xml contains the required SQLite JDBC driver and SLF4J logger dependencies:

Dependency Group	Artifact ID	Version	Purpose
org.xerial	sqlite-jdbc	3.46.0.0	SQLite database JDBC driver
org.slf4j	slf4j-api	2.0.16	SLF4J logging API abstraction
org.slf4j	slf4j-simple	2.0.16	SLF4J logger simple binding implementation

9.3 Manual JAR Installation (Non-Maven Setup)

If building without Maven in IntelliJ IDEA:

- 1. Download sqlite-jdbc-3.46.0.0.jar, slf4j-api-2.0.16.jar, and slf4j-simple-2.0.16.jar.
- 2. Navigate to File → Project Structure → Libraries → Add (+) → Java.
- 3. Select all three downloaded JAR files and apply settings.

9.4 Build & Execution Steps

1. Clone repository to your workspace or desktop.

2. Compile project: mvn clean compile

3. Seed SQLite Database: Run com.ug.optimizer.database.SimpleDBTest or DataSeeder.java.

4. Launch Application: Run com.ug.optimizer.Main.java to start the Console Menu UI.

9.5 Git Workflow & Team Commit Rules

• Feature Branching: Never commit directly to main. Create feature branches (e.g., feature/linked-list, feature/graph, feature/dijkstra).

• Conventional Commits: Use standard prefixes (feat:, fix:, docs:, test:, refactor:).

• Code Integrity: Write unit tests before opening Pull Requests.